DEVOPS HANDS-ON LAB GUIDE

Frontend + Backend on AWS EC2 with Nginx, AMI, Launch Templates, Load Balancers & Auto Scaling

A beginner-friendly, from-scratch guide: deploy a real 2-tier app across separate EC2 instances, connect it to MongoDB with a connection string, then put two load balancers and two Auto Scaling Groups in front, and prove it all works with live demos.

EC2 | Nginx | Node.js + Express | MongoDB Atlas | AMI | Launch Template | Target Groups | 2 × ALB | 2 × ASG | CloudWatch | systemd

Sample application: DevOps Task Board · Region used: ap-south-1 (Mumbai) · Ubuntu 24.04 · Node.js 22

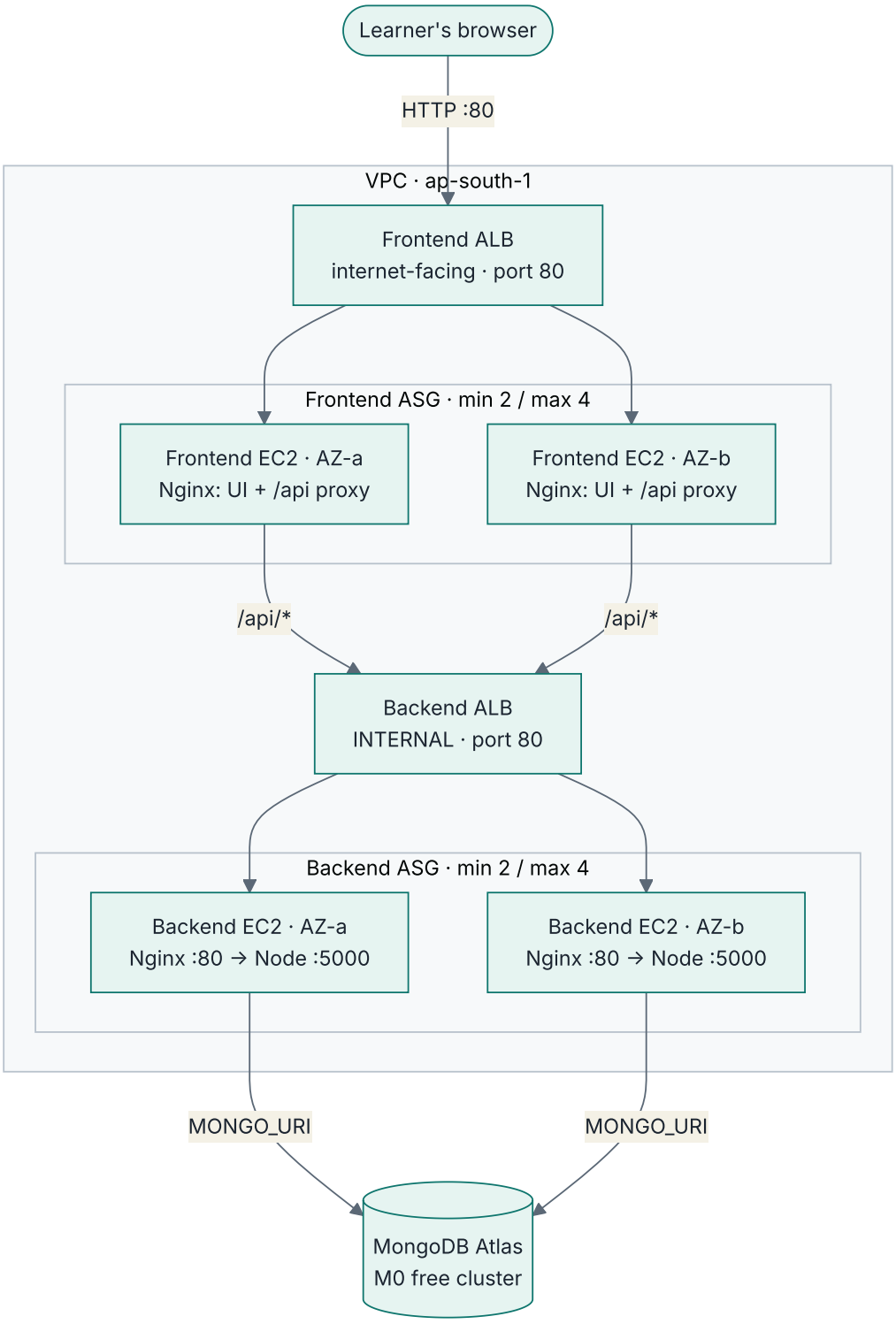
Contents

0. What you will build
1. Concepts from absolute scratch
2. How one request travels
3. Prerequisites
4. Part A: Run it on your laptop first, then push to GitHub
5. Part B: MongoDB Atlas (database + connection string)
6. Part C: Security groups (the firewall chain)
7. Part D: Backend EC2 #1 (by hand)
8. Part E: Frontend EC2 #1 (by hand)
9. Part F: Create AMIs (golden images)
10. Part G: Backend launch template + backend #2
11. Part H: Backend target group + internal Backend ALB
12. Part I and J: Frontend launch template, frontend #2, Frontend ALB
13. Part K: Auto Scaling Groups
14. Demos (prove that it works)
15. Troubleshooting
16. Cleanup (do it in THIS order)
17. Going to production (what's next)
18. Check your understanding
Appendix A: Repository file reference
Appendix B: API endpoints

0. What you will build

By the end of this lab you will have a real 2-tier application running on AWS:

- **2+ frontend servers** (EC2 + Nginx) behind an **internet-facing Application Load Balancer**
- **2+ backend servers** (EC2 + Nginx + Node.js) behind an **internal Application Load Balancer**
- A **MongoDB Atlas** database that every backend connects to with a **connection string**
- Two **Auto Scaling Groups** that add servers when CPU is high, remove them when it drops, and replace broken servers automatically
- New servers **register with the load balancer automatically**, and you will see it happen live in the app



What you will be able to show at the end

Demo	What learners see
Load balancing	The app's "Live load-balancing view" shows requests spread across 2 frontend and 2 backend instance IDs
Shared database	A task created through backend #1 is visible through backend #2

Demo	What learners see
Scale-out	Click "Generate CPU load", and 3 to 5 minutes later a new backend instance appears with a **NEW** badge
Auto-registration	The new instance shows up in the Target Group as *healthy* with no manual steps
Self-healing	Terminate a frontend instance and the ASG launches a replacement on its own
Scale-in	When CPU drops, the ASG goes back to 2 instances

Time and cost

- Time: about 3 to 4 hours the first time
- Cost: while everything runs (4 × t3.micro + 2 ALBs + public IPv4 addresses) it is roughly **US $0.15 to $0.25 per hour** in Mumbai. Check the AWS pricing pages for current numbers. **Always do the cleanup in Section 16.**

1. Concepts from absolute scratch

Read this section once before you touch AWS. Every later step refers back to it.

1.1 Frontend and backend

	Frontend	Backend
What	The part the user **sees**: HTML, CSS, JavaScript	The part that **does the work**: business logic, talks to the database
Runs where	Downloaded and run **inside the browser**	Runs **on the server** (Node.js process)
In our app	`frontend/index.html`, `style.css`, `app.js`	`backend/server.js` (Express API on port 5000)
Example	The "Add task" button	`POST /api/tasks` saves the task into MongoDB

Why separate them? They scale differently, can be deployed independently, and the database credentials stay on the backend, never in the browser.

1.2 How the browser talks to the backend in THIS app

This is the most important idea in the lab, and a very common point of confusion:

> The browser **never** talks to the backend directly. The JavaScript calls a **relative** URL like `/api/tasks`. That request goes to the **same server that served the page** (frontend Nginx). Nginx then **forwards** it to the backend.

Because of that:

- The backend address is **not hard-coded** in the frontend code
- There is **no CORS** problem (the browser only ever talks to one origin)
- The backend load balancer can be **internal** (no internet access at all), which is more secure

1.3 Networking basics

Term	Simple meaning	Why it matters here
Region	A geographic AWS location, e.g. `ap-south-1` (Mumbai)	Everything in this lab lives in one region
Availability Zone (AZ)	A separate data centre inside a region (`ap-south-1a`, `-1b`, `-1c`)	We spread servers over 2 AZs so one data-centre failure doesn't kill the app. An ALB needs **at least 2 AZs**
VPC	Your private network inside AWS	We use the **default VPC** to keep it simple
Subnet	A slice of the VPC inside one AZ	Each AZ in the default VPC has one public subnet

Term	Simple meaning	Why it matters here
Public IP	Reachable from the internet	Used for SSH and phase-1 testing
Private IP	Reachable only inside the VPC (e.g. `172.31.x.x`)	Servers talk to each other using private IPs
Port	A "door" number on a server: 22 = SSH, 80 = HTTP, 5000 = our Node app	Security groups open or close doors
Security Group (SG)	A virtual firewall attached to a server or ALB	We chain 4 SGs so each tier only accepts traffic from the tier in front of it

1.4 MongoDB and the connection string

MongoDB is a document database. It stores records as JSON-like documents. **MongoDB Atlas** is MongoDB hosted for you by MongoDB Inc.; the **M0** cluster is free.

The backend finds the database through a **connection string**, kept in an environment variable called `MONGO_URI`:

```
mongodb+srv://taskboard_user:MyPass123@cluster0.abcde.mongodb.net/taskboard?
retryWrites=true&w=majority
└protocol┘ └username┘ └password┘ └─────cluster host──────┘ └db┘
└────────options────────┘
```

Why an environment variable? The same code runs on your laptop, in Docker and on EC2. Only the `.env` file changes. Passwords never go into Git.

1.5 Nginx: why we use it on BOTH tiers

Nginx is a fast web server and **reverse proxy** (a server that receives requests and forwards them to another server).

Where	Job	Why
Frontend EC2	1) Serve `index.html`, `app.js`, `style.css`	Nginx serves static files far more efficiently than Node
	2) Forward `/api/*` to the Backend ALB	Browser only talks to one origin (no CORS), backend stays private
	3) Answer `/health` for the ALB	Cheap health check that doesn't touch the backend
Backend EC2	Listen on port 80 and forward `/api/*` to Node on `127.0.0.1:5000`	Node is never exposed directly. Nginx handles timeouts, body size limits, gzip and access logs, and is the standard place to add rate limiting or TLS later

> **Nginx and ALB DNS names (an important real-world detail):** an ALB's IP addresses change over time. If Nginx resolved the ALB name only once at start-up, it would keep sending traffic to old IPs and you'd get random **502** errors days later. Our config uses `resolver 169.254.169.253 valid=30s` (the Amazon DNS server) plus a variable in `proxy_pass`, so Nginx re-resolves the name every 30 seconds.

1.6 systemd

`systemd` is the Linux service manager. We create a **service** (`taskboard-api.service`) for the Node app.

Why? When the Auto Scaling Group launches a new server, nobody logs in to type `node server.js`. The app must **start on boot by itself** and **restart if it crashes**. systemd does both.

1.7 AMI (Amazon Machine Image)

An AMI is a **snapshot of a fully configured server**: OS + installed software + your code + config.

Why? You configure one server by hand once. Then every new server is launched from the AMI and is ready in about a minute, identical every time. This is the "golden image" pattern.

1.8 Launch Template

A Launch Template is a **saved recipe** for launching an instance: AMI + instance type + key pair + security group + user data.

Why? The Auto Scaling Group needs to know *exactly how* to create new servers. You also avoid clicking through the launch wizard again and again.

User data is a script inside the launch template that runs **once, as root, on the first boot** of every new instance. We use it for things that differ per environment, such as telling frontend Nginx the Backend ALB's address.

1.9 Target Group and Health Check

A **Target Group** is a list of servers (targets) that a load balancer can send traffic to. The ALB checks each target regularly with a **health check**:

Target group	Health check path	Healthy when
`taskboard-fe-tg`	`/health`	Frontend Nginx returns 200
`taskboard-be-tg`	`/api/health`	Backend Nginx → Node → **MongoDB connected** → 200

An unhealthy target gets **no traffic**. If the ASG uses ELB health checks, it is also **replaced**.

1.10 Application Load Balancer (ALB): why TWO?

An ALB spreads incoming HTTP requests across healthy targets in multiple AZs.

ALB	Scheme	Who calls it	Why
`taskboard-fe-alb`	**internet-facing**	Learners' browsers	One stable public URL for many frontend servers
`taskboard-be-alb`	**internal**	Only frontend Nginx	One stable private address for many backend servers. Not reachable from the internet at all

Why not just use backend private IPs? Private IPs change every time a server is replaced, and the ASG replaces servers all the time. The Backend ALB DNS name never changes, so frontend Nginx is configured once and works forever.

1.11 Auto Scaling Group (ASG)

An ASG keeps a **desired number** of instances running, between a **minimum** and **maximum**, using a launch template.

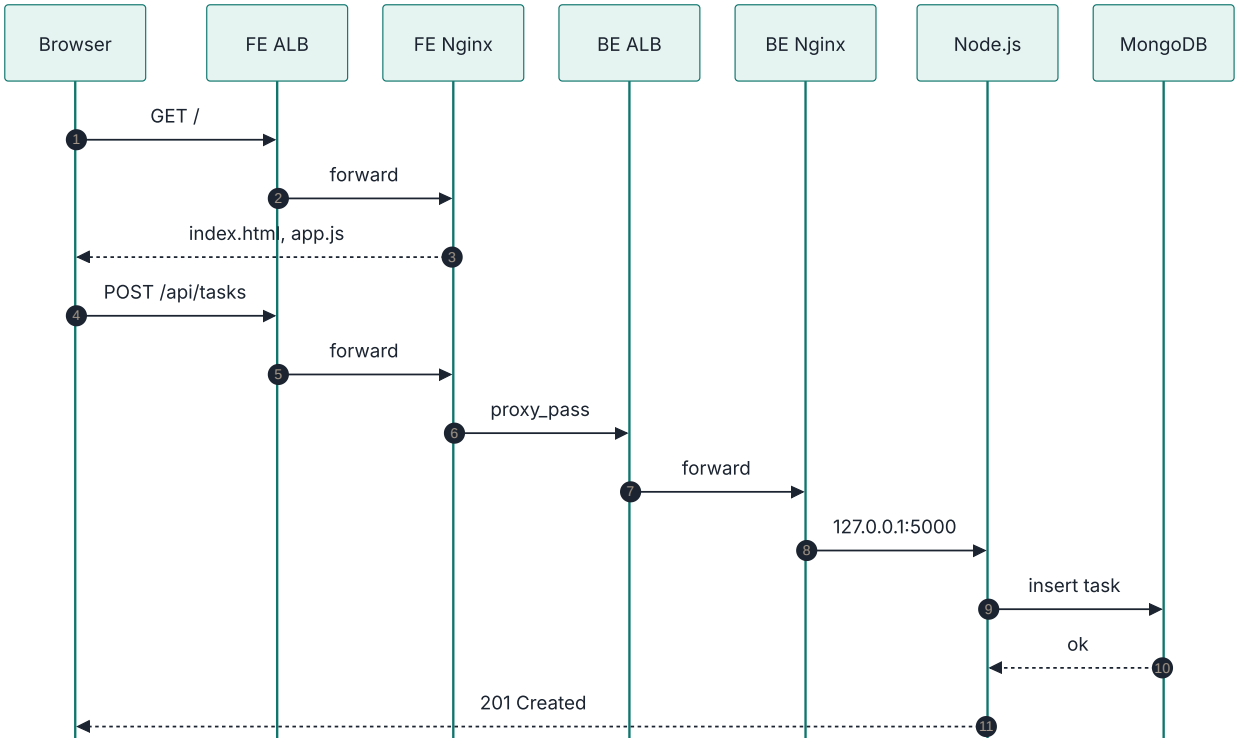
Setting	Our value	Meaning
Min	2	Never fewer than 2 (one per AZ = high availability)
Desired	2	Normal running count
Max	4	Cost safety limit
Scaling policy	Target tracking, Average CPU = 50%	Add servers when average CPU > 50%, remove them when it is well below
Health check	EC2 + **ELB**	Replace instances that fail the load balancer health check

When the ASG launches an instance it **registers it in the target group automatically**. That is the "auto-connect to the load balancer" you will demo.

1.12 Component cheat-sheet: what and WHY

Component	Why we use it
EC2	Virtual servers to run our frontend and backend
Nginx (frontend)	Serve static files + reverse proxy `/api` → no CORS, backend stays private
Nginx (backend)	Hide Node behind port 80, add timeouts, logs, limits
Node.js + Express	The REST API
MongoDB Atlas	Managed database, so every backend shares the same data
`.env` / `MONGO_URI`	Config outside the code, secrets out of Git
systemd	App starts on boot and restarts on crash. Required for auto scaling
AMI	Identical, ready-to-run servers in a minute
Launch Template	Recipe the ASG uses to create servers
User data	Per-environment setup on first boot
Target Group + health check	Know which servers are healthy
Frontend ALB (public)	One public URL, spreads traffic across frontends
Backend ALB (internal)	One stable private URL for backends
ASG	Scale out/in on CPU, self-heal, auto-register in target group
Security Groups (4)	Each tier accepts traffic only from the tier in front of it

2. How one request travels



3. Prerequisites

Need	Notes
AWS account	Use region **Asia Pacific (Mumbai)** `ap-south-1` (any region works; keep it the same everywhere)
MongoDB Atlas account	Free: https://www.mongodb.com/cloud/atlas/register
GitHub account	To host the code your EC2 servers will `git clone`
On your laptop	Git, Node.js LTS (https://nodejs.org), optional Docker Desktop, a terminal (PowerShell on Windows)
EC2 key pair	EC2 → Key Pairs → **Create key pair** → name `taskboard-key`, type RSA, format `.pem` → keep the downloaded file safe

> **Windows key-pair permission fix:** if `ssh` complains "UNPROTECTED PRIVATE KEY FILE", run in PowerShell:
>
> ```
> icacls .\taskboard-key.pem /inheritance:r
> icacls .\taskboard-key.pem /grant:r "$($env:USERNAME):(R)"
> ```

Names used in this guide

Use exactly these names and the screenshots and commands will match:

Resource	Name
Security groups	`taskboard-fe-alb-sg`, `taskboard-fe-sg`, `taskboard-be-alb-sg`, `taskboard-be-sg`
Instances (manual)	`taskboard-backend-1`, `taskboard-frontend-1`, `taskboard-backend-2`, `taskboard-frontend-2`
AMIs	`taskboard-backend-ami-v1`, `taskboard-frontend-ami-v1`
Launch templates	`taskboard-backend-lt`, `taskboard-frontend-lt`
Target groups	`taskboard-be-tg`, `taskboard-fe-tg`
Load balancers	`taskboard-be-alb` (internal), `taskboard-fe-alb` (internet-facing)
ASGs	`taskboard-be-asg`, `taskboard-fe-asg`

Placeholders you will replace

Placeholder	Example	Where you get it
`<YOUR_GITHUB_USERNAME>`	`chirag390`	Your GitHub profile
`<MONGO_URI>`	`mongodb+srv://...mongodb.net/taskboard?...`	Atlas → Connect (Section 5)

Placeholder	Example	Where you get it
`<BACKEND_PRIVATE_IP>`	`172.31.12.34`	EC2 → `taskboard-backend-1` → Private IPv4
`<BACKEND_ALB_DNS>`	`internal-taskboard-be-alb-123.ap-south-1.elb.amazonaws.com`	EC2 → Load balancers → `taskboard-be-alb`
`<FRONTEND_ALB_DNS>`	`taskboard-fe-alb-456.ap-south-1.elb.amazonaws.com`	EC2 → Load balancers → `taskboard-fe-alb`

The build order (and why this order)

Phase	Step	What you build	Section
1. By hand	1	Run on laptop + push to GitHub	4
	2	MongoDB Atlas + connection string	5
	3	4 security groups	6
	4	Backend EC2 #1 (Node + Nginx + systemd)	7
	5	Frontend EC2 #1 (Nginx) → **frontend ↔ backend ↔ MongoDB working**	8
2. Images, templates, LBs	6	Two AMIs	9
	7	Backend launch template + backend #2	10
	8	Backend target group + **internal** Backend ALB	11
	9	Frontend launch template + frontend #2	12
	10	Frontend target group + **internet-facing** Frontend ALB	12
3. Automation + proof	11	Two Auto Scaling Groups	13
	12	Demos	14

We build the **backend ALB before the frontend launch template** because the frontend's user data needs the Backend ALB DNS name.

4. Part A: Run it on your laptop first, then push to GitHub

Why first on the laptop? If something is broken in the code, you find out in 2 minutes on your laptop, not after 2 hours on AWS.

4.1 Get the code

Download or clone the project folder `devops-task-board`. Its structure:

```
devops-task-board/
├── frontend/            index.html, style.css, app.js
├── backend/             server.js, models/, lib/, .env.example, package.json
├── nginx/               frontend.conf.template, backend.conf.template
├── deploy/              systemd units, user-data scripts, setup scripts
├── docker-compose.yml   whole stack on your laptop
├── start-local.bat      Windows one-click launcher
└── docs/GUIDE.md        this guide
```

4.2 Option B: Node.js only (simplest, no Atlas needed yet)

Windows: double-click **`start-local.bat`**. Or in a terminal:

```
cd devops-task-board/backend
npm install
npm run dev:local
```

It starts a **temporary in-memory MongoDB** and the API, and Node also serves the frontend files. Open http://localhost:5000.

4.3 Option A: Docker Compose (same shape as AWS)

```
cd devops-task-board
docker compose up -d --build
```

Open http://localhost:8080. Here the chain is the same as on AWS: `browser → frontend Nginx container → backend Nginx container → Node container → Mongo container`.

Load-balancing preview on your laptop:

```
docker compose up -d --scale api=3
```

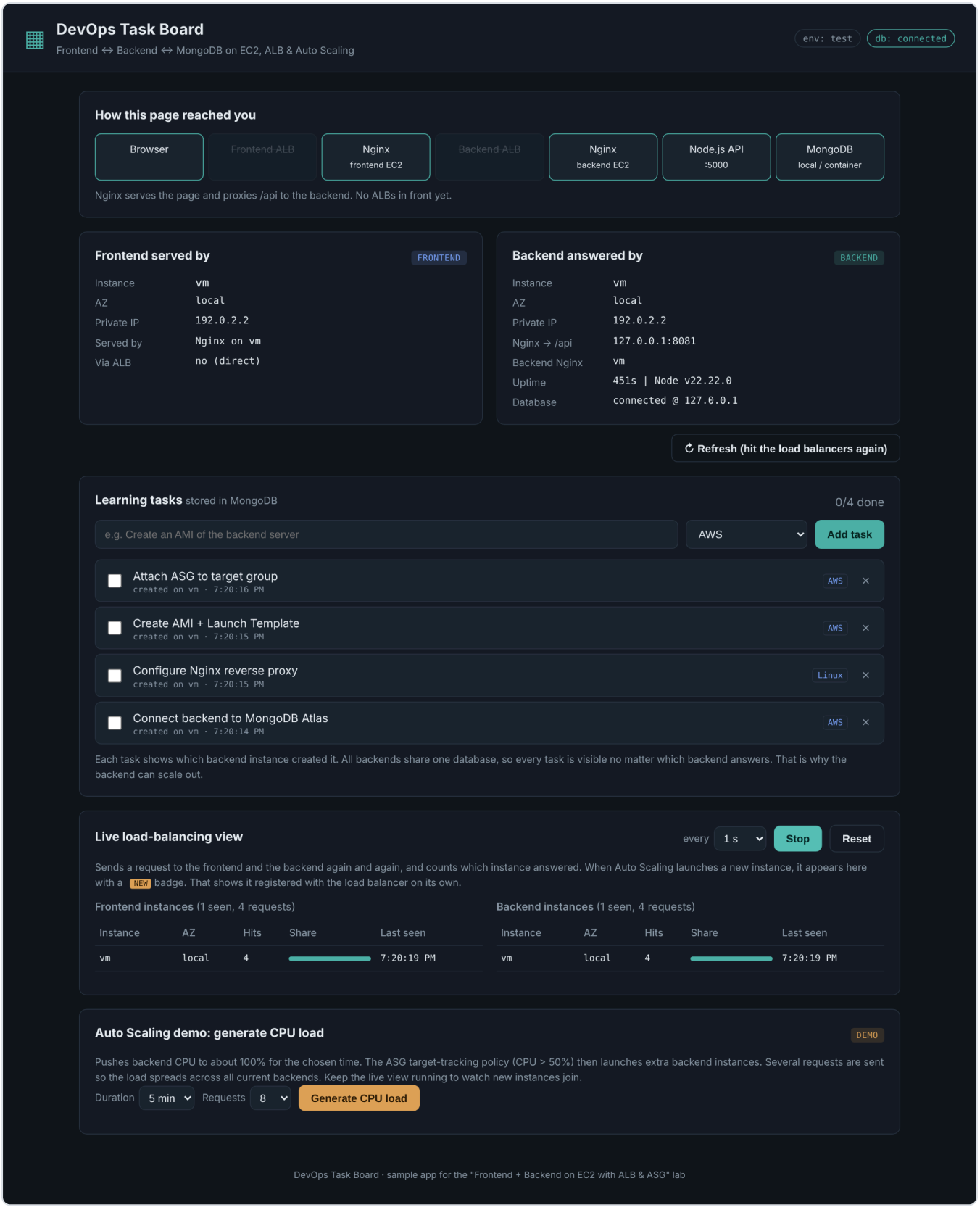
In the app press **Start** in "Live load-balancing view". You will see 3 backend container IDs sharing the traffic.

4.4 Option C: Your laptop + MongoDB Atlas

After Section 5 (Atlas) you can also test the real connection string locally:

```
cd backend
cp .env.example .env          # Windows: copy .env.example .env
# edit .env: MONGO_URI=<your Atlas string>, SERVE_FRONTEND=true
npm start                     # http://localhost:5000
```

4.5 Tour of the app



Panel	What it shows
How this page reached you	Lights up each hop that was really used (ALBs, Nginx, Node, MongoDB)
Frontend served by / Backend answered by	Instance ID, AZ, private IP, uptime, DB status of the servers that answered
Learning tasks	CRUD stored in MongoDB. Each task records which backend created it
Live load-balancing view	Polls every second and counts hits per instance. New instances get a **NEW** badge
Auto Scaling demo	Pushes backend CPU to about 100% to trigger scale-out

4.6 Push to your own GitHub

1. GitHub → **New repository** → name `devops-task-board` → **Public** (so EC2 can clone without a token) → do **not** add a README → Create.
2. On your laptop:

```
cd devops-task-board
git init
git add .
git commit -m "DevOps Task Board - initial version"
git branch -M main
git remote add origin https://github.com/<YOUR_GITHUB_USERNAME>/devops-task-board.git
git push -u origin main
```

> **Check:** `.env` must **not** appear on GitHub. `.gitignore` already excludes it.
>
> **Why `.gitattributes` ?** It forces Linux line endings (LF) for `.sh` , `.service` and `.template` files. If you edit them on Windows, CRLF endings would break bash and systemd on EC2.

5. Part B: MongoDB Atlas (database + connection string)

1. Log in to Atlas → **Create** a project named `taskboard`.
2. **Create a cluster** → choose **M0 (Free)** → Provider **AWS** → Region **Mumbai (ap-south-1)** → name `Cluster0` → **Create Deployment**.
3. **Database user** (the wizard asks right away, or go to *Security → Database Access*):
   - Username `taskboard_user`, click **Autogenerate Secure Password** and copy it.
   - Role: *Read and write to any database* → **Create Database User**.
   - **Why a separate user?** The app should never use your Atlas login. Each app gets its own least-privilege DB user.
4. **Network Access** (*Security → Network Access → Add IP Address*):
   - Click **Allow access from anywhere** → `0.0.0.0/0` → Confirm.
   - **Why anywhere?** Every instance the ASG launches gets a **new public IP**, so you can't list them in advance. Security still comes from the username and password, and TLS encryption is always on.
   - **Production way:** private subnets + a **NAT Gateway with an Elastic IP** (allow only that one IP), or **VPC peering / PrivateLink** (paid Atlas tiers).
5. **Get the connection string:** Cluster → **Connect** → **Drivers** → Driver *Node.js* → copy:

```
mongodb+srv://taskboard_user:<db_password>@cluster0.xxxxx.mongodb.net/?
retryWrites=true&w=majority&appName=Cluster0
```

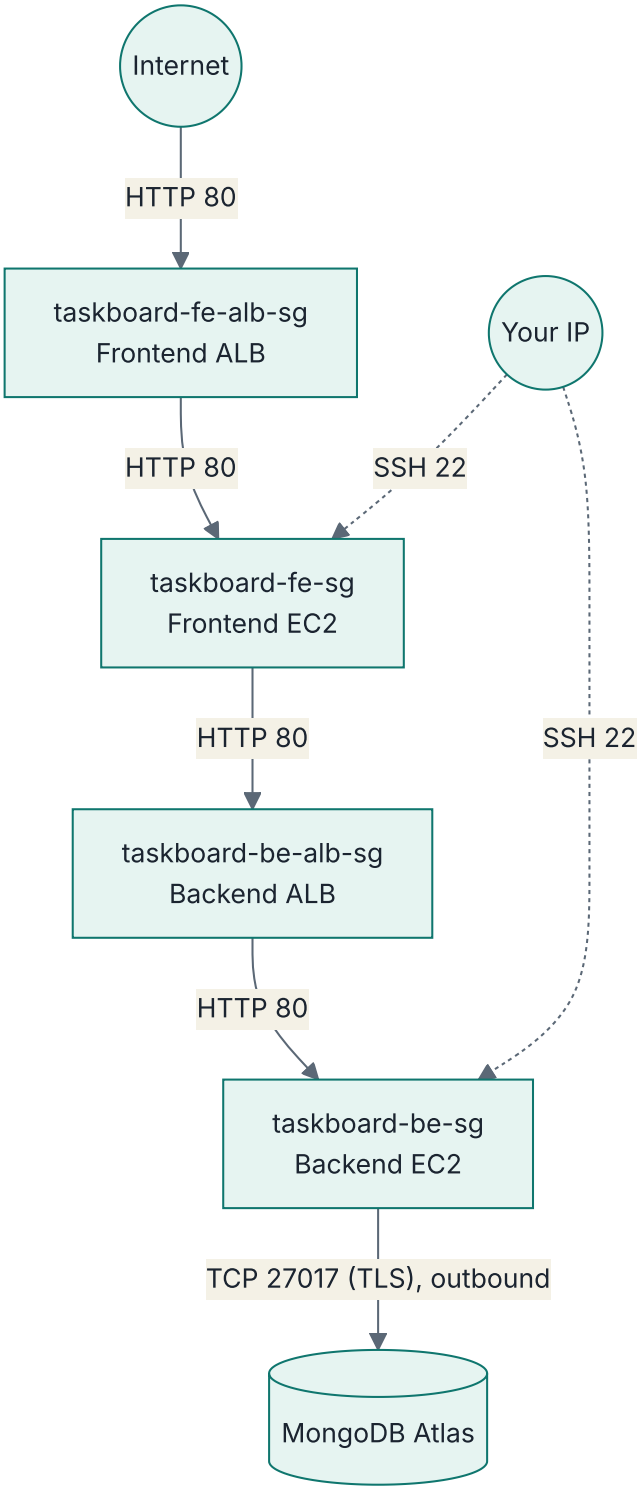
6. Make it yours:
   - Replace `<db_password>` with the real password.
   - Add the **database name** `taskboard` right after `.net/`:

```
mongodb+srv://taskboard_user:REALPASS@cluster0.xxxxx.mongodb.net/taskboard?
retryWrites=true&w=majority&appName=Cluster0
```

   - If the password contains special characters like `@ : / ? # %`, URL-encode them (`@` → `%40`, `#` → `%23`), or just regenerate a password with only letters and numbers.

> **Check (optional):** put this string in `backend/.env` on your laptop (Section 4.4) and run `npm start`. You should see `MongoDB connected -> host: ...mongodb.net`.

6. Part C: Security groups (the firewall chain)



Why chain security groups by referencing other SGs instead of IPs? A rule like "allow port 80 from `taskboard-fe-sg`" automatically covers **every instance** in that SG, including ones the ASG creates later with brand-new IPs.

Go to **EC2 → Security Groups → Create security group** (VPC: default). Create them **in this order**, because a rule can only reference an SG that already exists:

#	Name	Inbound rules	Why
1	`taskboard-fe-alb-sg`	HTTP 80 from `0.0.0.0/0`	Anyone on the internet can open the app
2	`taskboard-fe-sg`	HTTP 80 from `taskboard-fe-alb-sg`	Only the frontend ALB reaches frontend servers
		HTTP 80 from **My IP** *(temporary, phase 1)*	So you can test frontend #1 directly before the ALB exists
		SSH 22 from **My IP**	You can log in to configure
3	`taskboard-be-alb-sg`	HTTP 80 from `taskboard-fe-sg`	Only frontend Nginx can call the backend ALB
4	`taskboard-be-sg`	HTTP 80 from `taskboard-be-alb-sg`	Only the backend ALB reaches backend servers
		HTTP 80 from `taskboard-fe-sg` *(temporary, phase 1)*	Frontend #1 talks to backend #1 directly before the ALB exists
		SSH 22 from **My IP**	You can log in to configure

Leave **Outbound** as default (*All traffic → 0.0.0.0/0*). Servers need it to download packages, clone from GitHub and reach MongoDB Atlas.

> Port **5000 is never opened**. Node.js is only reachable from inside the backend server through Nginx.

7. Part D: Backend EC2 #1 (by hand)

7.1 Launch the instance

EC2 → **Launch instance**:

Field	Value	Why
Name	`taskboard-backend-1`	
AMI	**Ubuntu Server 24.04 LTS** (64-bit x86)	Popular, well documented
Instance type	`t3.micro`	Free-tier eligible in most accounts, enough for the lab
Key pair	`taskboard-key`	For SSH
Network settings → Edit	VPC default, Subnet **ap-south-1a**, Auto-assign public IP **Enable**	Public IP for SSH + outbound internet
Firewall	*Select existing security group* → `taskboard-be-sg`	
Storage	8 GiB gp3 (default)	

Click **Launch instance**. Wait until *Instance state = Running* and *Status check = 2/2 passed*.

7.2 Connect with SSH

Copy the **Public IPv4 address**, then from the folder that has your key:

```
ssh -i taskboard-key.pem ubuntu@<BACKEND_PUBLIC_IP>
```

(Or use **EC2 Instance Connect** in the console: select instance → Connect.)

7.3 Install software

```
# 1. Update the package list and install Nginx, Git, curl and envsubst
sudo apt-get update -y
sudo apt-get install -y nginx git curl gettext-base

# 2. Install Node.js 22 LTS from NodeSource (Ubuntu's own package is too old)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs

# 3. Check versions
node -v      # v22.x
npm -v
nginx -v
```

Package	Why
`nginx`	Reverse proxy in front of Node

Package	Why
`git`	Clone our code from GitHub
`gettext-base`	Gives us `envsubst`, which fills `${VARIABLES}` into our Nginx templates
`nodejs`	Runs the backend

7.4 Get the code and install dependencies

```
sudo git clone https://github.com/<YOUR_GITHUB_USERNAME>/devops-task-board.git /opt/taskboard
sudo chown -R ubuntu:ubuntu /opt/taskboard     # our service runs as the ubuntu user
cd /opt/taskboard/backend
npm install --omit=dev                         # production dependencies only
```

Why `/opt/taskboard`? `/opt` is the Linux convention for add-on applications, and a fixed path lets the systemd unit and scripts find the code.

7.5 Configure the connection string (.env)

```
cp .env.example .env
nano .env
```

Set:

```
PORT=5000
MONGO_URI=<MONGO_URI>              # your full Atlas string from Section 5
APP_ENV=aws
ENABLE_STRESS=true
SERVE_FRONTEND=false               # on EC2 Nginx serves the frontend, not Node
```

Save with `Ctrl+O`, Enter, `Ctrl+X`. Protect it: `chmod 600 .env`

7.6 Test-run the app by hand

```
node server.js
```

Expected:

```
Running on: i-0abc123... (ap-south-1a, 172.31.x.x) [ec2]
Task Board API listening on port 5000
Connecting to MongoDB: mongodb+srv://taskboard_user:****@cluster0...
MongoDB connected -> host: ac-xxxx-shard-00-01.xxxxx.mongodb.net, db: taskboard
```

Notice that the app found its own **instance ID** via the EC2 metadata service (IMDSv2) and **masked the password** in the log.

Stop it with `Ctrl+C`.

> If MongoDB does not connect, see Section 15 (Troubleshooting) before continuing.

7.7 Run it as a systemd service

```
sudo cp /opt/taskboard/deploy/taskboard-api.service /etc/systemd/system/
sudo systemctl daemon-reload                # tell systemd about the new file
sudo systemctl enable --now taskboard-api   # start now AND on every boot
sudo systemctl status taskboard-api --no-pager
curl http://localhost:5000/api/health
# {"status":"ok","db":"connected","instanceId":"i-0abc..."}
```

Follow logs live: `sudo journalctl -u taskboard-api -f`

7.8 Configure Nginx in front of Node

```
cd /opt/taskboard
API_UPSTREAM=127.0.0.1:5000 DNS_RESOLVER=169.254.169.253 \
  envsubst '${API_UPSTREAM} ${DNS_RESOLVER}' \
  < nginx/backend.conf.template | sudo tee /etc/nginx/sites-available/taskboard

sudo ln -sf /etc/nginx/sites-available/taskboard /etc/nginx/sites-enabled/taskboard
sudo rm -f /etc/nginx/sites-enabled/default     # remove the Nginx welcome page
sudo nginx -t                                    # ALWAYS test before restarting
sudo systemctl enable nginx
sudo systemctl restart nginx
```

What each line does:

- `envsubst` takes the template and fills in `${API_UPSTREAM}` and `${DNS_RESOLVER}`. We list the two variables explicitly so Nginx's own variables like `$host` are left untouched.
- `sites-available` holds configs, and a symlink in `sites-enabled` turns one on (Ubuntu's convention).
- `nginx -t` checks the syntax. Never restart a production Nginx without it.

Test the full backend chain **through Nginx on port 80**:

```
curl http://localhost/api/health          # Nginx -> Node -> MongoDB
curl -i http://localhost/api/info         # note the X-Backend-Nginx header
curl http://localhost/                    # {"error":"backend serves /api/* only"}
```

Note the **Private IPv4** of this instance (EC2 console). This is your `<BACKEND_PRIVATE_IP>`.

> **Shortcut:** `deploy/setup-backend.sh` does Sections 7.3 to 7.8 in one go. Do it by hand the first time so you understand each step.

8. Part E: Frontend EC2 #1 (by hand)

8.1 Launch

Same as 7.1, but:

Field	Value
Name	`taskboard-frontend-1`
Subnet	**ap-south-1b** (a different AZ from the backend, for practice)
Security group	`taskboard-fe-sg`

8.2 Install and configure

SSH in: `ssh -i taskboard-key.pem ubuntu@<FRONTEND_PUBLIC_IP>`

```
sudo apt-get update -y
sudo apt-get install -y nginx git curl gettext-base

# Code
sudo git clone https://github.com/<YOUR_GITHUB_USERNAME>/devops-task-board.git /opt/taskboard

# Static files -> Nginx web root
sudo mkdir -p /var/www/taskboard
sudo cp -r /opt/taskboard/frontend/. /var/www/taskboard/

# instance.json generator: runs at EVERY boot, writes this server's ID/AZ/IP
sudo cp /opt/taskboard/deploy/taskboard-meta.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now taskboard-meta
cat /var/www/meta/instance.json
```

Why generate `instance.json` at every boot? When we create an AMI from this server, the file inside it holds **this** server's ID. Every new server launched from the AMI must write **its own** ID, so the app can show which frontend answered.

8.3 Nginx: static files + reverse proxy to backend #1

```
cd /opt/taskboard
BACKEND_HOST=<BACKEND_PRIVATE_IP> DNS_RESOLVER=169.254.169.253 \
  envsubst '${BACKEND_HOST} ${DNS_RESOLVER}' \
  < nginx/frontend.conf.template | sudo tee /etc/nginx/sites-available/taskboard

sudo ln -sf /etc/nginx/sites-available/taskboard /etc/nginx/sites-enabled/taskboard
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl enable nginx && sudo systemctl restart nginx
```

Why the **private** IP? The two servers are in the same VPC. Private traffic stays inside AWS, is free within an AZ (and cheap across AZs), and is allowed by our SG rule *80 from* `taskboard-fe-sg`.

Test:

```
curl http://localhost/health              # ok   (frontend Nginx itself)
curl http://localhost/api/health          # {"status":"ok","db":"connected",...}  (through to
backend)
```

8.4 Phase 1 checkpoint: open it in your browser

Open `http://<FRONTEND_PUBLIC_IP>`.

- "Frontend served by" shows `taskboard-frontend-1`'s instance ID (AZ `ap-south-1b`)
- "Backend answered by" shows `taskboard-backend-1`'s instance ID (AZ `ap-south-1a`)
- In "How this page reached you", the two ALBs are **crossed out** (not built yet)
- Add a task, and it is saved in MongoDB Atlas (you can see it in Atlas → Browse Collections → `taskboard.tasks`)

Frontend ↔ Backend ↔ MongoDB is working across 2 EC2 instances.

9. Part F: Create AMIs (golden images)

Why now? Both servers are fully configured and tested. We freeze them so every future server is an exact copy.

For **each** of `taskboard-backend-1` and `taskboard-frontend-1`:

1. EC2 → Instances → select the instance → **Actions → Image and templates → Create image**
2. Name: `taskboard-backend-ami-v1` / `taskboard-frontend-ami-v1`
3. Description: `Task Board backend v1 (Node 22, Nginx, systemd)` (and similar for the frontend)
4. Leave *Reboot instance* **checked**, which ensures a consistent file system snapshot
5. **Create image**

Go to **EC2 → AMIs** and wait until *Status = Available* (about 3 to 8 minutes).

What is inside each AMI:

Backend AMI	Frontend AMI
Ubuntu + Node 22 + Nginx	Ubuntu + Nginx
`/opt/taskboard` code + `node_modules`	`/var/www/taskboard` static files
`backend/.env` with `MONGO_URI`	`taskboard-meta.service` (enabled)
`taskboard-api.service` (enabled)	Nginx config (pointing to backend-1, which user data will change)
Nginx reverse-proxy config (enabled)	

> **Warning:** The backend AMI contains your `MONGO_URI` password. Never make the AMI public. In production you'd store it in **AWS Secrets Manager / SSM Parameter Store** and read it in user data with an IAM role.

10. Part G: Backend launch template + backend #2

10.1 Create the backend launch template

EC2 → **Launch Templates → Create launch template**:

Field	Value	Why
Name	`taskboard-backend-lt`	
Auto Scaling guidance	Check *Provide guidance to help me set up a template that I can use with EC2 Auto Scaling*	Highlights the fields the ASG needs
AMI	*My AMIs* → `taskboard-backend-ami-v1`	Our golden image
Instance type	`t3.micro`	
Key pair	`taskboard-key`	So you can still SSH in for debugging
Network settings → Subnet	**Don't include in launch template**	The ASG chooses subnets across AZs
Security groups	`taskboard-be-sg`	
Advanced details → Detailed CloudWatch monitoring	**Enable**	1-minute CPU metrics, so scaling reacts in a few minutes instead of 10+ (small extra cost)
Advanced details → User data	paste the content of `deploy/user-data-backend.sh`	Restarts services and self-tests on first boot

Create launch template.

10.2 Launch backend #2 from the template

Launch Templates → select `taskboard-backend-lt` → **Actions → Launch instance from template**:

- Number of instances: **1**
- Network settings → Subnet: **ap-south-1b**
- Resource tags → `Name` = `taskboard-backend-2`
- **Launch instance**

No SSH, no installation: it is ready in about a minute. Verify from **frontend-1**:

```
curl http://<BACKEND_2_PRIVATE_IP>/api/health
# {"status":"ok","db":"connected","instanceId":"i-0<backend-2-id>"}
```

The instance ID is **different** from backend-1, but it uses the **same database**. That's the AMI + launch template working.

11. Part H: Backend target group + internal Backend ALB

11.1 Target group

EC2 → **Target Groups → Create target group**:

Field	Value
Target type	**Instances**
Name	`taskboard-be-tg`
Protocol / Port	HTTP / **80** (backend Nginx)
VPC	default
Health check path	**`/api/health`**
Advanced health check	Healthy threshold **2**, Unhealthy threshold **2**, Timeout **5**, Interval **10**, Success codes **200**

Why `/api/health` and not `/`? It checks the whole chain: Nginx → Node → **MongoDB**. If a backend loses its DB connection it returns 503, the ALB stops sending it traffic, and the ASG replaces it.

Why these numbers? Lower thresholds and intervals make the demos faster (healthy in about 20 seconds). The defaults are slower and fine for production.

Next → **Register targets**: select `taskboard-backend-1` and `taskboard-backend-2` → **Include as pending below** → **Create target group**.

After creating: target group → **Attributes → Edit** → *Deregistration delay* = **30** seconds (default 300). This makes scale-in and termination demos quicker.

11.2 Internal load balancer

EC2 → **Load Balancers → Create load balancer → Application Load Balancer**:

Field	Value	Why
Name	`taskboard-be-alb`	
Scheme	**Internal**	Never reachable from the internet
IP address type	IPv4	
VPC	default	
Mappings	**ap-south-1a**, **ap-south-1b** (and 1c)	ALB needs at least 2 AZs
Security groups	remove *default*, add `taskboard-be-alb-sg`	
Listener	HTTP : 80 → Forward to `taskboard-be-tg`	

Create load balancer. Wait for *State = Active*. Copy the **DNS name**. That is `<BACKEND_ALB_DNS>`, which looks like `internal-taskboard-be-alb-1234567890.ap-south-1.elb.amazonaws.com`.

Target group → **Targets** tab: both backends should become **healthy**.

11.3 Test the internal ALB from frontend-1

From your laptop the internal ALB will **time out**, which is correct because it is private. Test from inside the VPC, on **frontend-1**:

```
BE=<BACKEND_ALB_DNS>
for i in 1 2 3 4 5 6; do curl -s http://$BE/api/info | grep -o '"instanceId":"[^"]*"'; done
```

You should see the two backend instance IDs **alternating**. That's load balancing.

11.4 Point frontend-1 at the Backend ALB

```
cd /opt/taskboard
BACKEND_HOST=<BACKEND_ALB_DNS> DNS_RESOLVER=169.254.169.253 \
  envsubst '${BACKEND_HOST} ${DNS_RESOLVER}' \
  < nginx/frontend.conf.template | sudo tee /etc/nginx/sites-available/taskboard
sudo nginx -t && sudo systemctl reload nginx
```

Refresh `http://<FRONTEND_PUBLIC_IP>` a few times. "Backend answered by" now changes between the two backends, and *Backend ALB* is lit in the request path.

12. Part I and J: Frontend launch template, frontend #2, Frontend ALB

12.1 Prepare the frontend user data

Open `deploy/user-data-frontend.sh` and replace the placeholder:

```
BACKEND_HOST="internal-taskboard-be-alb-1234567890.ap-south-1.elb.amazonaws.com"
```

Why user data here? The AMI still points Nginx to `<BACKEND_PRIVATE_IP>` (backend-1). User data rewrites the Nginx config on first boot so **every new frontend** talks to the **Backend ALB**. If the ALB ever changes, you only edit the launch template, not the AMI.

12.2 Frontend launch template

Same as 10.1 with:

Field	Value
Name	`taskboard-frontend-lt`
AMI	`taskboard-frontend-ami-v1`
Security group	`taskboard-fe-sg`
Detailed monitoring	Enable
User data	your edited `deploy/user-data-frontend.sh`

12.3 Launch frontend #2

Launch instance from template → Subnet **ap-south-1a** → Name `taskboard-frontend-2`. Open `http://<FRONTEND_2_PUBLIC_IP>`: it works straight away and shows its **own** instance ID.

Debug a new instance's user data if needed:

```
sudo cat /var/log/taskboard-user-data.log
sudo tail -50 /var/log/cloud-init-output.log
```

12.4 Frontend target group

Field	Value
Name	`taskboard-fe-tg`
Protocol / Port	HTTP / 80
Health check path	**`/health`**
Thresholds	2 / 2, timeout 5, interval 10
Targets	`taskboard-frontend-1` , `taskboard-frontend-2`

Then **Attributes → Deregistration delay = 30**.

Why `/health` (Nginx only) for the frontend? If it called the backend, one backend problem would make **all** frontends look unhealthy, and the ASG would kill perfectly good frontends. Keep each tier's health check about **that tier**.

12.5 Frontend ALB (internet-facing)

Field	Value
Name	`taskboard-fe-alb`
Scheme	**Internet-facing**
Mappings	ap-south-1a, ap-south-1b (and 1c)
Security group	`taskboard-fe-alb-sg`
Listener	HTTP:80 → `taskboard-fe-tg`

When *Active*, open `http://<FRONTEND_ALB_DNS>`.

All seven boxes in "How this page reached you" are lit. Press **Start** in the live view. You should see **2 frontend** and **2 backend** instances sharing requests.

2 frontends + 2 backends behind 2 load balancers.

13. Part K: Auto Scaling Groups

Now we hand the servers over to AWS so it can scale and heal them.

13.1 Backend ASG

EC2 → **Auto Scaling Groups → Create Auto Scaling group**:

Step 1: Name `taskboard-be-asg` · Launch template `taskboard-backend-lt` · Version **Latest**

Step 2: Network: VPC default · Subnets **ap-south-1a, ap-south-1b, ap-south-1c** *Why several AZs?* The ASG spreads instances evenly across them.

Step 3: Load balancing:

- **Attach to an existing load balancer** → *Choose from your load balancer target groups* → `taskboard-be-tg`
- Health checks: **Turn on Elastic Load Balancing health checks**
- Health check grace period: **120** seconds

Why ELB health checks? By default the ASG only replaces an instance if the **VM** is broken. With ELB checks, it also replaces instances whose **app** is broken (failing `/api/health`).

Step 4: Group size and scaling:

- Desired **2** · Min **2** · Max **4**
- Automatic scaling: **Target tracking scaling policy**
  - Metric type: **Average CPU utilization**
  - Target value: **50**
  - Instance warmup: **120** seconds

Step 5 (Notifications): skip. **Step 6 Tags:** Key `Name`, Value `taskboard-be-asg`, so the instances get this name.

Create Auto Scaling group.

13.2 Frontend ASG

Repeat with: name `taskboard-fe-asg`, template `taskboard-frontend-lt`, target group `taskboard-fe-tg`, same sizes and policy, tag `Name = taskboard-fe-asg`.

13.3 Watch the ASGs register instances

- ASG → **Activity** tab: "Launching a new EC2 instance: i-0..." (×2 for each ASG)
- **Target groups → Targets**: each target group now has **4 targets** (2 manual + 2 from the ASG)

13.4 Retire the hand-made instances

The ASGs now own the fleet. Servers you launched by hand are "pets"; ASG servers are "cattle", which can be replaced any time.

1. Wait until the 2 new ASG instances are **healthy** in *both* target groups.
2. EC2 → Instances → select `taskboard-backend-1`, `taskboard-backend-2`, `taskboard-frontend-1`, `taskboard-frontend-2` → **Instance state → Terminate**.
3. The target groups show them **draining** for 30 seconds, then they disappear. The app keeps working, and the live view shows only the ASG instances.

13.5 Remove the temporary phase-1 rules

Security group	Delete this inbound rule
`taskboard-fe-sg`	HTTP 80 from My IP
`taskboard-be-sg`	HTTP 80 from `taskboard-fe-sg`

Now the only way in is through the load balancers.

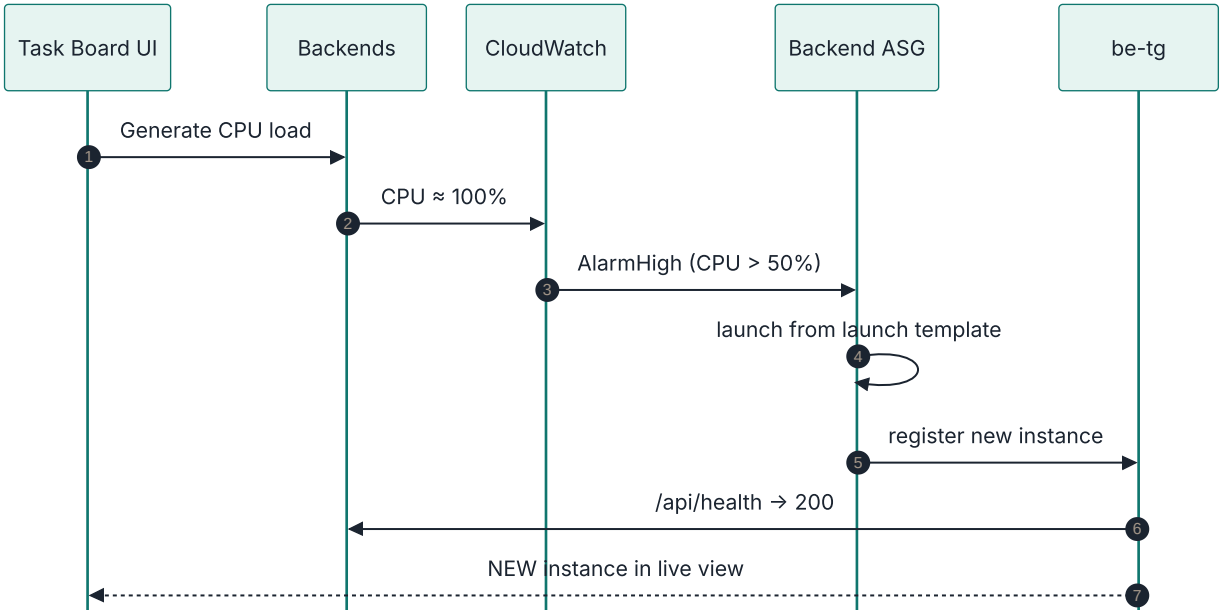
14. Demos (prove that it works)

Open `http://<FRONTEND_ALB_DNS>` and press **Start** in "Live load-balancing view" (1 s). Keep it running for every demo.

Demo 1: Load balancing + shared database

1. Watch the live view: hits are split across **2 frontend** and **2 backend** instances.
2. Add a few tasks. Each one shows `created on i-0...`. Different tasks come from different backends, but **all** tasks show up every time.
3. **Why?** Backends keep no data locally (they're *stateless*); everything is in MongoDB. That's what makes it safe to add or remove backends at any time.

Demo 2: Scale-out on CPU (backend)



1. In the app: **Auto Scaling demo** → Duration **5 min**, Requests **8** → **Generate CPU load**. The output lists which backends are now busy.
2. Watch in the AWS console:
   - **CloudWatch → Alarms**: `TargetTracking-taskboard-be-asg-AlarmHigh-...` goes to **In alarm** (about 3 minutes)
   - **ASG → Activity**: "Launching a new EC2 instance..." · *Desired capacity* goes from 2 to 3 or 4
   - **Target groups → taskboard-be-tg → Targets**: a new target appears, *initial* → **healthy**
   - **App live view**: a new backend ID appears with a **NEW** badge
3. **This proves the new instance was configured by the AMI + launch template and connected to the load balancer by the ASG, with zero manual steps.**

Alternative way to create load (SSH to a backend):

```
sudo apt-get install -y stress-ng
stress-ng --cpu 0 --timeout 300s     # 0 = all vCPUs, for 5 minutes
```

Demo 3: Scale-out (frontend)

Frontend Nginx barely uses CPU, so generate load directly on the frontends. Through ASG → Instance management, find each frontend's public IP. SSH is allowed from your IP:

```
sudo apt-get install -y stress-ng && stress-ng --cpu 0 --timeout 300s
```

Run it on both frontends. In 3 to 5 minutes `taskboard-fe-asg` launches a new frontend, it registers in `taskboard-fe-tg`, and the **live view's frontend table** shows a NEW instance.

Quick manual alternative: ASG → **Edit** → Desired capacity **3** → Update. Watch the new instance join the target group and the live view.

Demo 4: Self-healing

a) Instance dies: EC2 → terminate one `taskboard-fe-asg` instance. → The target drains → ASG Activity shows the instance was taken out of service → a replacement launches → it appears in the live view with **NEW**.

b) App dies but VM is fine: SSH to one backend and stop the app:

```
sudo systemctl stop nginx
```

→ `/api/health` fails → the target becomes **unhealthy** (about 20 s) → the ALB stops sending it traffic. The app keeps working on the other backend → after the grace period the ASG (ELB health check) **terminates and replaces** it.

Demo 5: Scale-in

After the load stops, CPU drops. About 15 minutes later the `AlarmLow` alarm fires and the ASG terminates the extra instances (back to **2**). The live view marks them `gone?` (faded), and the target group shows them *draining* first.

> **Why does scale-in take longer than scale-out?** AWS adds capacity quickly (to protect users) and removes it slowly (to avoid flapping up and down).

15. Troubleshooting

Symptom	Likely cause	How to check / fix
`ssh: connect ... timed out`	SG has no port 22 from your **current** IP	Edit the SG → SSH → *My IP* again (home IPs change)
`UNPROTECTED PRIVATE KEY FILE` (Windows)	Key file permissions	`icacls` commands in Section 3
`MongoDB connection failed: bad auth`	Wrong user or password in `MONGO_URI`	Re-check Atlas → Database Access, URL-encode special characters
`MongoDB connection failed: ... timed out` / `Server selection timed out`	Atlas Network Access doesn't allow the IP	Atlas → Network Access → `0.0.0.0/0`
`querySrv ENOTFOUND`	Typo in the cluster host	Copy the string again from Atlas → Connect
UI shows "Database not connected"	Same as above	`sudo journalctl -u taskboard-api -n 50`
`/api/health` returns 503	Node is running but DB is disconnected	Check MongoDB as above
Browser **502 Bad Gateway** on the whole page	No healthy frontend targets	`taskboard-fe-tg` → Targets → reason; on the instance: `sudo systemctl status nginx`
502 only on `/api` calls	Frontend Nginx can't reach the backend: wrong `BACKEND_HOST`, backend Nginx or Node down	Frontend: `sudo tail /var/log/nginx/error.log`; backend: `curl localhost/api/health`
504 Gateway Timeout on `/api`	A security group blocks the hop (packets dropped)	Check SG chain (Section 6): `be-alb-sg` must allow 80 from `fe-sg`, `be-sg` 80 from `be-alb-sg`
Backend targets **unhealthy**	Wrong path/port, Nginx or Node not running, DB down	Path must be `/api/health`, port 80; on the instance: `curl -i localhost/api/health`
Internal ALB times out from laptop	Expected, it is **internal**	Test from a frontend instance
New ASG instance not working	User data failed or placeholder left in it	`/var/log/taskboard-user-data.log`, `/var/log/cloud-init-output.log`
`bash\r: No such file` / `$'\r': command not found`	Windows CRLF line endings in a script	`.gitattributes` prevents it; fix with `sudo sed -i 's/\r$//' file.sh`
`nginx: [emerg] no resolver defined`	Template not rendered with `DNS_RESOLVER`	Re-run the `envsubst` command with both variables

Symptom	Likely cause	How to check / fix
Scaling doesn't happen	Detailed monitoring off, max reached, alarm still *insufficient data*, `ENABLE_STRESS=false`	CloudWatch → Alarms; launch template → monitoring; the app shows a 403 if stress is disabled
Live view shows only one backend	Only one healthy target, or one ASG instance still warming up	Target group → Targets

Useful commands:

```
sudo systemctl status taskboard-api nginx --no-pager
sudo journalctl -u taskboard-api -f            # backend app logs
sudo tail -f /var/log/nginx/access.log         # every request through Nginx
sudo tail -f /var/log/nginx/error.log          # proxy errors (502/504 reasons)
sudo nginx -t                                  # validate config
curl -i http://localhost/api/health            # backend self-test
cat /var/www/meta/instance.json                # frontend identity
```

16. Cleanup (do it in THIS order)

Resources that depend on others must go first:

#	Delete	Where	Note
1	`taskboard-fe-asg`, `taskboard-be-asg`	EC2 → Auto Scaling Groups → Delete	Terminates their instances
2	`taskboard-fe-alb`, `taskboard-be-alb`	EC2 → Load Balancers → Delete	ALBs bill per hour
3	`taskboard-fe-tg`, `taskboard-be-tg`	EC2 → Target Groups	
4	`taskboard-frontend-lt`, `taskboard-backend-lt`	EC2 → Launch Templates	
5	Both AMIs → **Deregister**	EC2 → AMIs	
6	Their **snapshots**	EC2 → Snapshots	Snapshots keep billing after the AMI is gone!
7	Any leftover instances	EC2 → Instances	e.g. the manual ones if not terminated
8	SGs: `be-sg` → `be-alb-sg` → `fe-sg` → `fe-alb-sg`	EC2 → Security Groups	Reverse order because of references
9	(Optional) Atlas cluster, key pair		M0 is free to keep

17. Going to production (what's next)

Lab shortcut	Production practice
Public subnets for servers	**Private subnets** + NAT Gateway; only ALBs in public subnets
Atlas allows `0.0.0.0/0`	Allow only the NAT Elastic IP, or VPC peering / PrivateLink
`MONGO_URI` baked in the AMI	**Secrets Manager / SSM Parameter Store** + IAM instance role
HTTP only	**HTTPS** on the frontend ALB with a free **ACM** certificate, HTTP → HTTPS redirect
ALB DNS name	Custom domain with **Route 53** alias record
Manual AMI per release	New AMI → new launch template version → **ASG Instance refresh** (rolling update), or build AMIs with Packer
Click-ops	Everything as **Terraform** / CloudFormation
SSH for debugging	**SSM Session Manager** (no port 22 at all)

Releasing a new version of the app (rolling update): update code → update one instance (or build a new one) → create `...-ami-v2` → launch template → **Create new version** with the new AMI → ASG → **Instance refresh** (min healthy 50%). The ASG replaces instances a few at a time with zero downtime.

18. Check your understanding

1. Why can the Backend ALB be **internal** even though the app runs in the learner's browser? *Because the browser only calls the frontend (`/api/...` on the same origin); frontend Nginx calls the Backend ALB from inside the VPC.*
2. Why do we use a **variable + resolver** in Nginx `proxy_pass`? *ALB IPs change; this makes Nginx re-resolve the DNS name every 30 seconds instead of caching it forever.*
3. What would break if the backend stored tasks in memory instead of MongoDB? *Each backend would have different data; tasks would "disappear" depending on which backend answers, and be lost on scale-in.*
4. Why is the frontend health check `/health` and not `/api/health`? *So a backend problem doesn't make healthy frontends look broken and get replaced.*
5. Why reference security groups instead of IP addresses? *ASG instances get new IPs; an SG reference automatically includes every current and future member.*
6. What is the difference between the AMI and the launch template? *The AMI is the disk image (OS + software + code). The launch template is the recipe (AMI + type + SG + key + user data + monitoring).*
7. Why does the ASG need ELB health checks turned on? *Without them it only notices dead VMs, not a dead app.*
8. Why is `systemctl enable` essential for auto scaling? *New instances must start the app on boot with no human logging in.*

Appendix A: Repository file reference

File	Used where	Purpose
`backend/server.js`	Backend	Express API, health check, IMDSv2 instance detection, stress endpoint
`backend/.env.example`	Backend	Template for `.env` (`MONGO_URI`, `PORT`, …)
`frontend/index.html`, `app.js`, `style.css`	Frontend	UI, calls relative `/api/...` URLs
`nginx/frontend.conf.template`	Frontend	Static files + `/api` reverse proxy + `/health`
`nginx/backend.conf.template`	Backend	Port 80 → `127.0.0.1:5000`
`deploy/taskboard-api.service`	Backend	systemd unit for Node
`deploy/taskboard-meta.service` + `write-instance-meta.sh`	Frontend	Writes `/var/www/meta/instance.json` at every boot
`deploy/user-data-backend.sh`	Backend LT	Restart + self-test on first boot
`deploy/user-data-frontend.sh`	Frontend LT	Points Nginx at `<BACKEND_ALB_DNS>`
`deploy/setup-backend.sh`, `setup-frontend.sh`	First instances	Optional one-shot setup
`docker-compose.yml`	Laptop	Same architecture in containers
`start-local.bat`	Windows laptop	One-click local run

Appendix B: API endpoints

Method	Path	Purpose
GET	`/api/health`	200 when API + DB OK (ALB health check)
GET	`/api/info`	Instance ID, AZ, private IP, uptime, DB status
GET	`/api/tags`	Allowed task tags
GET / POST	`/api/tasks`	List / create tasks
PATCH / DELETE	`/api/tasks/:id`	Update / delete a task
POST	`/api/stress?seconds=300`	Burn CPU on that backend (needs `ENABLE_STRESS=true`)
GET	`/health` (frontend Nginx)	Frontend ALB health check
GET	`/instance.json` (frontend Nginx)	Which frontend served the page